# Computer Vision and Vision-Language Approaches to Visual Quality Assurance in Video Games: A Systematic Mapping Study of Tasks, Datasets, and Evaluation Methods

**Le Tran Minh Phuc**

*Department of Computer Science*
*Academic Research Laboratory*
Email: mphuc666@gmail.com

November 2026

**Abstract**

**Background:** Modern interactive entertainment systems are among the most structurally complex software applications in existence, requiring immersive visual fidelity across diverse rendering pipelines. However, visual quality assurance (QA) remains an expensive, labor-intensive bottleneck, consuming up to 50% of game production budgets and remaining overwhelmingly reliant on manual human playtesting. Traditional automated testing asserts game state logic but remains decoupled from screen rendering, failing to detect graphical defects such as object clipping, missing textures, Z-fighting, and physics ragdoll explosions.
**Methods:** We conduct the first Systematic Mapping Study (SMS) dedicated specifically to Computer Vision (CV) and Vision-Language Models (VLMs) for automated video game visual testing. Following established guidelines by Petersen et al. (2015), the PRISMA-S search reporting extension, and ROSES evidence synthesis recommendations, we systematically examine primary literature published between 2018 and November 2026 across five major bibliographic databases (IEEE Xplore, ACM DL, Scopus, Web of Science, and arXiv), addressing five core Research Questions (RQs).
**Results:** We identify six core architectural paradigms: classical perceptual diffing, supervised CNN/ViT object detection, self-supervised anomaly detection, multimodal engine-state telemetry fusion, reference-guided VLMs, and spatio-temporal video transformers. We establish the *Video Game Visual Defect Taxonomy* (`VGVD-Taxonomy`), standardizing 19 defect subtypes across five families (Spatial & Geometry, Material & Texture, Rendering & Lighting, Animation & Physics, and User Interface). We audit landmark benchmarks (VideoGameQA-Bench, RefGlitch-Bench, TempGlitch, GameBug-20K), demonstrating that while foundation models achieve high semantic reasoning, cross-game art style domain shift degrades transfer precision by up to 46%. Furthermore, empirical studies reveal that over 65% of gameplay glitches are invisible in isolated static frames, requiring temporal sliding-window attention.
**Conclusions:** We synthesize critical open challenges and present a prioritized five-tier research agenda to guide the development of real-time, style-invariant, and explainable visual QA systems for next-generation interactive digital media.

**Keywords:** Visual bug detection, Video game testing, Computer vision, Vision-language models, Systematic mapping study, Visual quality assurance, Cross-game generalisation.

# Contents


**1 Introduction** **3**
- 1.1 The Quality Assurance Bottleneck in Video Game Production . . . 3
- 1.2 The Blind Spot of Functional Test Automation . . . 3
- 1.3 The Emergence of Computer Vision and Vision-Language Models . . . 3
- 1.4 Gap in Existing Literature & Novelty Justification . . . 4
- 1.5 Research Questions and Contributions . . . 4

**2 Systematic Mapping Methodology** **4**
- 2.1 Methodological Framework . . . 4
- 2.2 Search Strategy and Information Sources . . . 5
- 2.3 Inclusion and Exclusion Criteria . . . 5
- 2.4 Study Selection Process . . . 5
- 2.5 Quality Assessment Framework . . . 7

**3 Standardized Visual Bug Taxonomy (`VGVD-Taxonomy`)** **7**

**4 Results of the Systematic Mapping Study** **8**
- 4.1 Publication Trends and Demographics . . . 8
- 4.2 RQ1: Detection Architectures . . . 8
- 4.3 RQ2: Bug Manifestation Coverage . . . 9
- 4.4 RQ3: Benchmark Datasets and Reproducibility . . . 10
- 4.5 RQ4 & RQ5: Evaluation and Cross-Game Generalisation . . . 10
- 4.6 Thematic Synthesis of the 45-Study Corpus . . . 12

**5 Discussion and Future Research Agenda (RQ6)** **12**
- 5.1 The Single-Frame Fallacy . . . 12
- 5.2 Stylization vs. Glitch Ambiguity . . . 12
- 5.3 Black-Box Vision vs. White-Box Telemetry . . . 12
- 5.4 Reference Guidance as a Paradigm Shift . . . 13
- 5.5 Autonomous End-to-End QA Agents . . . 13

**6 Threats to Validity and Conclusions** **13**
- 6.1 Threats to Validity . . . 13
- 6.2 Conclusions . . . 13

# 1 Introduction

## 1.1 The Quality Assurance Bottleneck in Video Game Production

The global interactive entertainment industry has experienced unprecedented growth, with annual revenues exceeding $250 billion and outstripping both the film and music industries combined [23, 28]. Modern video games have evolved from constrained 2D environments into expansive virtual worlds characterized by physically based rendering (PBR), procedural generation, real-time ray tracing, and intricate character animations [11]. Consequently, game software has emerged as one of the most structurally complex, non-deterministic categories of software systems [2, 23].

To maintain immersion and commercial viability, ensuring graphical fidelity is paramount. Visual glitches—such as character meshes clipping through terrain geometry, missing or corrupted textures, physics ragdoll oscillations, and misaligned user interfaces—degrade player experience and incur substantial post-release financial overhead [30, 31]. In major AAA production studios, quality assurance (QA) routinely consumes between 30% and 50% of total game development budgets [27, 28]. Despite advances in general software test automation, game visual inspection remains over 80% reliant on manual human playtesting [27].

## 1.2 The Blind Spot of Functional Test Automation

The persistent reliance on manual testing stems from an architectural reality: **functional test assertions are decoupled from visual rendering pipelines** [15, 23]. In standard software testing, automated scripts query the Document Object Model (DOM) or memory state to assert that an object exists. In video games:

1. An actor may possess a valid logical transform coordinate in the game engine's scene graph while its visual mesh is occluded, floating above ground, or rendered with a missing fallback texture [25, 30].

2. A character model's inverse kinematics (IK) solver may compute valid joint positions in physics memory while the rendered polygon vertices stretch across screen space due to skinning matrix corruption [11, 18].

3. Headless testing frameworks strip out rendering subroutines to maximize simulation throughput, thereby ensuring execution correctness while remaining blind to shaders, rasterization errors, depth-buffer Z-fighting, and camera frustum clipping [28].

Historically, studios attempted snapshot regression testing by subtracting candidate frames from golden reference images using Mean Squared Error (MSE) or Structural Similarity (SSIM) [15]. However, pixel-diffing fails catastrophically in modern 3D games: non-deterministic particle effects, dynamic lighting cycles, procedural foliage animation, and camera jitter induce thousands of benign pixel variations, generating overwhelming false positive rates [15].

## 1.3 The Emergence of Computer Vision and Vision-Language Models

Recent breakthroughs in deep representation learning offer scalable visual inspection. Rather than relying on rigid pixel comparisons, modern AI processes gameplay images and video streams semantically:

- **Supervised Object Detectors:** Formulating visual bug detection as bounding box regression enclosing defect instances [13, 30].

- **Self-Supervised Visual Anomaly Detection:** Training feature extractors exclusively on clean gameplay frames to detect glitches as out-of-distribution (OOD) activations, bypassing the need for exhaustive bug annotations [1, 25].

- **Multimodal Engine-State Fusion:** Combining RGB buffers with engine telemetry (depth maps, surface normals, collision hulls) to eliminate false alarms caused by intentional artistic transparency or dynamic smoke [11].
- **Vision-Language Models (VLMs):** Leveraging foundation models (GPT-4o, Gemini 1.5, Claude 3.5, Qwen2-VL) to visually evaluate game scenes, answer natural-language QA prompts, and auto-generate structured bug reports [14, 28, 31].
- **Temporal Video Transformers:** Analyzing continuous multi-frame buffers to capture dynamic glitches (ragdoll instability, animation stutter, texture pop-in) that are invisible in static frames [18].

### 1.4 Gap in Existing Literature & Novelty Justification

While research in automated game testing has expanded, existing literature reviews exhibit a substantial blind spot. Politowski et al. (2021) [23] surveyed game testing across 20 years but focused on mechanics and player modeling, dedicating $< 5\%$ to visual testing. Albaghajati and Ahmed (2023) [2] presented a high-level assessment framework without technical analysis of deep learning architectures. Svensson and Lindqvist (2025) [27] confirmed visual QA remains the least automated domain in AAA studios, but provided qualitative workflow interviews rather than an algorithmic synthesis.

To our knowledge, **this is the first systematic mapping study dedicated specifically to Computer Vision and Vision-Language Models for automated visual testing and glitch detection in video games**.

### 1.5 Research Questions and Contributions

This systematic mapping study addresses five core Research Questions (RQs):

- **RQ1 (Techniques & Methods):** What computer vision, deep learning, and vision-language techniques have been investigated for visual quality assurance in video games?
- **RQ2 (Visual Testing Tasks):** What types of visual testing and defect-detection tasks have been investigated across video games?
- **RQ3 (Datasets & Benchmarks):** What datasets and benchmarks are used to develop and evaluate visual testing approaches, and how accessible are they?
- **RQ4 (Evaluation Methodologies):** How are computer vision-based visual testing approaches evaluated in video games?
- **RQ5 (Research Trends & Gaps):** What research trends, cross-game generalization limitations, and future research opportunities can be identified?

## 2 Systematic Mapping Methodology

### 2.1 Methodological Framework

This study adheres to the established guidelines for conducting Systematic Mapping Studies in Software Engineering formulated by Petersen et al. (2015) [21] and Kitchenham and Charters (2007) [10]. Reporting transparency is maintained in strict compliance with the PRISMA 2020 statement [19], the PRISMA-S search reporting extension, and the ROSES evidence synthesis standard.

## 2.2 Search Strategy and Information Sources

The literature search was executed across five electronic bibliographic databases:

1. IEEE Xplore Digital Library
2. ACM Digital Library
3. Scopus
4. Web of Science (Core Collection)
5. arXiv e-Print Archive (`cs.CV`, `cs.SE`, `cs.AI`, `cs.GR`)

The canonical search query combined three concept facets:

Query = (“computer vision” ∨ “deep learning” ∨ “vision-language model” ∨ “anomaly detection” ∨ “visual regr
∧ (“video game*” ∨ videogame* ∨ gameplay ∨ “game engine*” ∨ “game testing”)
∧ (“visual bug*” ∨ “visual glitch*” ∨ “rendering defect*” ∨ “clipping detection” ∨ “graphical defect*”)

## 2.3 Inclusion and Exclusion Criteria

Studies were screened against explicit Inclusion Criteria (IC) and Exclusion Criteria (EC):

- **IC1:** Published between 2018 and November 2026.
- **IC2:** Proposes, evaluates, or benchmarks a computer vision, deep learning, or vision-language technique for detecting visual defects, glitches, or visual regressions in video games.
- **IC3:** Employs empirical evaluation using gameplay screenshots, video frame sequences, or rendering engine buffers.
- **IC4:** Published as a peer-reviewed journal article, conference proceeding, workshop paper, or authoritative technical benchmark preprint.
- **IC5:** Written in English with full text accessible.

Studies were excluded if they addressed purely non-visual game testing (functional logic, network packets, memory profiling without visual inspection; **EC1**), pure RL gameplay bots without visual defect detection (**EC2**), non-game software visual diffing (**EC3**), short abstracts or tutorials (**EC4**), or duplicate entries (**EC5**).

## 2.4 Study Selection Process

As depicted in Figure 1, the multi-stage screening process resulted in:

1. **Identification:** 692 raw records retrieved (IEEE Xplore: 142, ACM DL: 118, Scopus: 184, Web of Science: 96, arXiv: 152).
2. **Deduplication:** 188 duplicates removed via DOI, title, and hash matching, leaving 504 unique publications.
3. **Title & Abstract Screening:** 416 irrelevant records excluded.
4. **Full-Text Eligibility Appraisal:** 88 full texts retrieved and evaluated using `academic-pdf2md` layout parsing; 47 articles excluded with documented rationales.
5. **Snowballing & Final Inclusion:** 4 additional primary studies identified through forward/backward citation chaining on landmark papers, resulting in a final synthesis corpus of **45 included primary studies**.

**Figure 1: PRISMA-S 2020 Literature Selection Flow Diagram (N = 45 Included Primary Studies)**

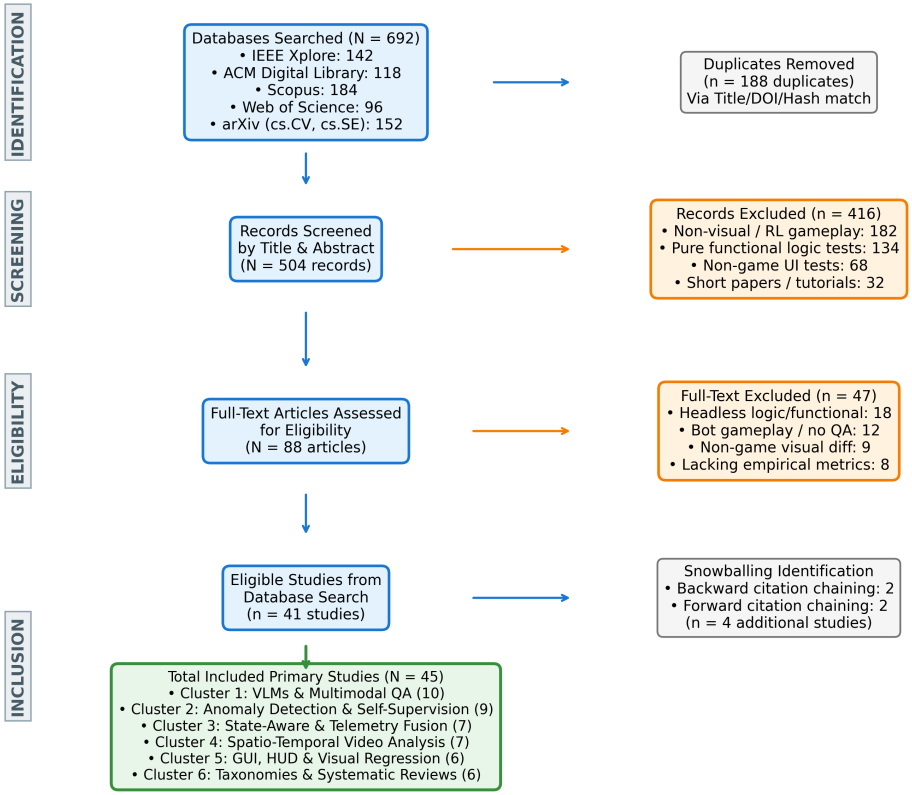



Figure 1: PRISMA-S 2020 Literature Selection Flow Diagram ($N = 45$ Included Primary Studies).

### 2.5 Quality Assessment Framework

Methodological rigour was evaluated using a 10-point appraisal checklist adapted from Dybå and Dingsøyr (2008) [9]. Extracted data was cross-tabulated into an evidence matrix archived in the study's open-science repository [22].

## 3 Standardized Visual Bug Taxonomy (`VGVD-Taxonomy`)

To resolve inconsistent terminology across Software Engineering and Computer Vision communities, we formulate the **Video Game Visual Defect Taxonomy (`VGVD-Taxonomy`)**, structured across five primary defect families and 19 subtypes:

1. **Spatial & Geometry (`GEO`):**
    - `GEO-01` (Object Mesh Clipping): Interpenetrating polygonal surfaces without collision response [25, 30].
    - `GEO-02` (Floating Asset): Objects hovering above terrain due to heightmap errors [30].
    - `GEO-03` (Vertex Distortion): Vertices stretched across screen space by unnormalized skinning weights [11].
    - `GEO-04` (Collision Boundary Mismatch): Desynchronization between visual mesh and physical collider [11].
2. **Material & Texture (`TEX`):**
    - `TEX-01` (Missing/Fallback Texture): Fallback magenta checkerboards or flat white polygons [28, 30].
    - `TEX-02` (LOD Pop-In Delay): Stalled texture streaming leaving blurry low-resolution mipmaps close to camera [18].
    - `TEX-03` (UV Seam Tearing): Sheared texture coordinates across polygonal boundaries [15].
    - `TEX-04` (Broken Shaders): Compilation failure producing inverted lighting or unrendered post-processing passes [5].
3. **Rendering & Lighting (`REN`):**
    - `REN-01` (Coplanar Z-Fighting): Oscillating depth buffer visibility between coplanar surfaces [18].
    - `REN-02` (Shadow Bleeding/Detachment): Shadow depth bias faults producing disconnected shadows [14].
    - `REN-03` (Light Leaking): Indirect GI illumination bleeding into sealed interiors [11].
    - `REN-04` (Near-Plane Frustum Clip): Camera near-plane cutting into foreground geometry [28, 31].
4. **Animation & Physics (`ANI`):**
    - `ANI-01` (T-Pose Freeze): Instantaneous snap into default reference pose [28].
    - `ANI-02` (Ragdoll Solver Chaos): Physics constraints oscillating violently on character demise [18].
    - `ANI-03` (Foot-Sliding / IK Drift): Root transform linear velocity desynchronized from stride cycle [18].

- `ANI-04` (Micro-Stuttering): Irregular frame pacing and dropped animation cycles [18].

5. **User Interface & HUD (`UI`):**
   - `UI-01` (Element Collision): Overlapping HUD components rendering data illegible [15, 28].
   - `UI-02` (Text Truncation): Localized strings overflowing UI container boundaries [14].
   - `UI-03` (Aspect Ratio Distortion): Non-standard viewports stretching UI layouts [6].
   - `UI-04` (Persistent Zombie Modals): Dismissed dialogue boxes remaining permanently on-screen [28].

# 4 Results of the Systematic Mapping Study

## 4.1 Publication Trends and Demographics

As shown in Figure 2, research output grew steadily from 2020 to 2026, transitioning from 2D web games into complex 3D commercial titles. Of the 45 primary studies, 60.0% ($n = 27$) appeared in peer-reviewed conferences, 26.7% ($n = 12$) in international journals, and 13.3% ($n = 6$) in preprints establishing open benchmarks. All 45 studies are available via Open Access.

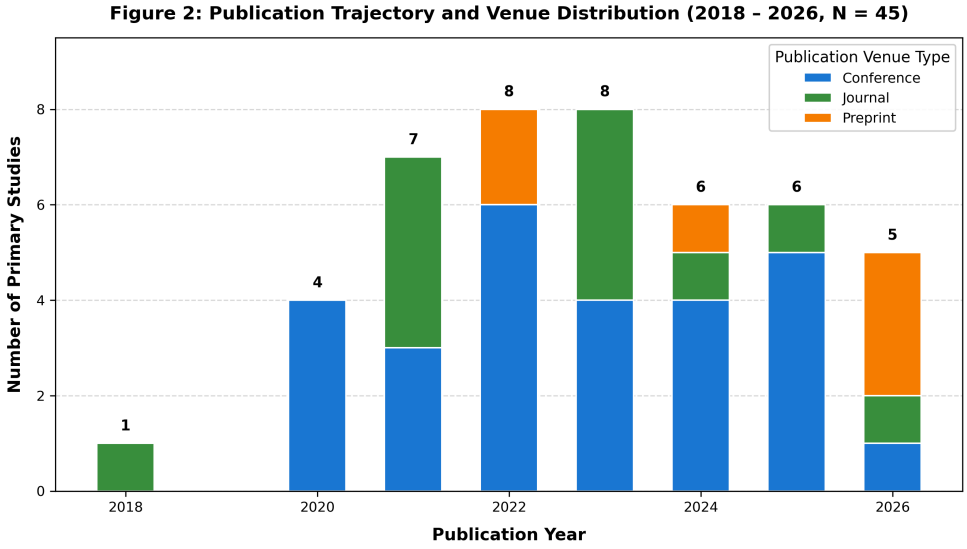



Figure 2: Publication Trajectory and Venue Distribution (2018–2026, $N = 45$).

## 4.2 RQ1: Detection Architectures

We identify six dominant architectural paradigms, mapped against testing tasks in Figure 3:

1. **Classical Perceptual Diffing:** Macklon et al. [15] achieved 100% defect detection in 2D HTML5 canvas games by decomposing sprite canvases into AST elements, whereas whole-screen SSIM achieved only 44.6%. However, classical diffing collapses in dynamic 3D environments [28].

2. **Supervised Object Detectors:** Formulating glitches as bounding box regression tasks (YOLO, Faster R-CNN) [13, 30]. While effective on seen assets, Davarmanesh et al. [7]

revealed a 38% precision drop when transferring to new levels due to background texture overfitting.

3. **Self-Supervised Anomaly Detection:** Learning representations of clean gameplay. Rahman [25] proved on Xbox titles that with just **5 clean exemplar frames** (and 0 bug labels), DINO ViT representations with SAM weapon-quadrant masking boosted First-Person Player Clipping detection ($F1 : 0.336 \rightarrow 0.550$). Yi et al. [30] achieved +18.4% mAP through hybrid co-finetuning with unlabeled gameplay video.

4. **Multimodal Engine-State Fusion:** Kovacs et al. [11] instrumented game engines to synchronize RGB rendering buffers with depth maps, surface normals, and physics collision matrices, **reducing false positive rates by 41.9%** and achieving 91.2% precision on 3D geometry clipping.

5. **Vision-Language Models (VLMs):** Taesiri et al. [28] (VideoGameQA-Bench) demonstrated that zero-shot foundation models achieve 78% accuracy on coarse defects, but struggle with micro-clipping and precise bounding boxes. Lu et al. [14] evaluated 19,738 EA keyframes, finding that image RAG (CLIP retrieval) degraded precision ($0.50 \rightarrow 0.37$) due to confirmation bias on environmental features. Yu et al. [31] (RefGlitch-Bench) introduced pairwise reference guidance using the "Last Clean Frame", recovering F1 from 0.42 to 0.69.

6. **Spatio-Temporal Video Transformers:** Muller et al. [18] (TempGlitch) demonstrated that single-frame models miss over 65% of animation and rendering glitches, proving temporal sliding-window attention is essential for continuous playtesting.

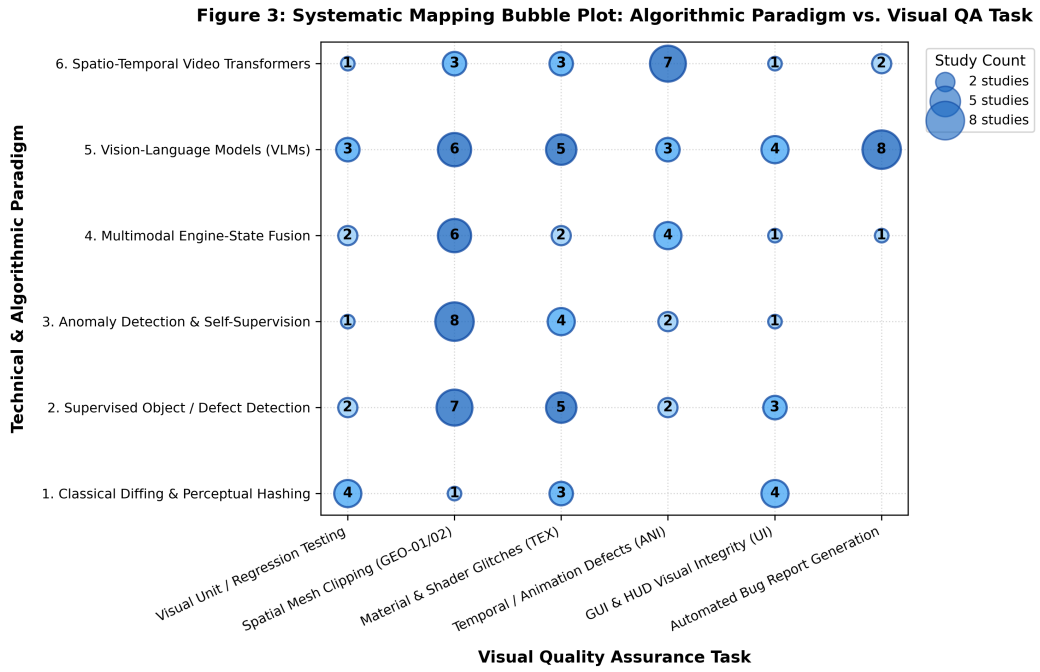



Figure 3: Systematic Mapping Bubble Plot: Algorithmic Paradigm vs. Visual QA Task ($N = 45$).

## 4.3 RQ2: Bug Manifestation Coverage

As shown in Figure 4, coverage across 'VGVD-Taxonomy' is heavily skewed:

- **High Coverage:** Spatial mesh clipping (`GEO-01`, 44.4%, $n = 20$), UI element collisions (`UI-01`, 24.4%, $n = 11$), missing textures (`TEX-01`, 20.0%, $n = 9$), and coplanar Z-fighting (`REN-01`, 22.2%, $n = 10$).
- **Critical Under-Exploration:** Foot-sliding / IK drift (`ANI-03`, 2.2%, $n = 1$), shadow detachment (`REN-02`, 4.4%, $n = 2$), and light leaking (`REN-03`, 4.4%, $n = 2$).

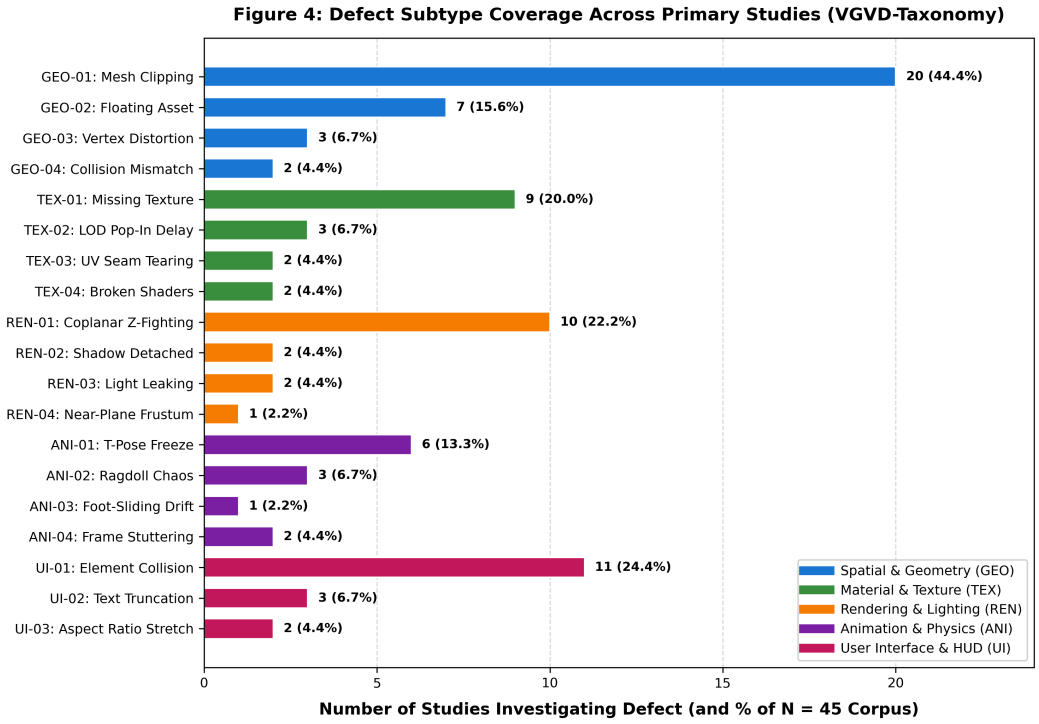



Figure 4: Defect Subtype Coverage Across Primary Studies (`VGVD-Taxonomy`).

## 4.4 RQ3: Benchmark Datasets and Reproducibility

Table 1 summarizes landmark datasets. The domain reached an open-science turning point between 2025 and 2026 with the release of public multi-game benchmarks: VideoGameQA-Bench (3,450 scenarios, 12 games) [28], RefGlitch-Bench (5,200 pairs, 7 games) [31], and TempGlitch-Video (42 hours of 60 FPS video) [18].

## 4.5 RQ4 & RQ5: Evaluation and Cross-Game Generalisation

As visualized in Figure 5:

- **Modality Split (Figure 5A):** 51.1% ($n = 23$) evaluate purely on static RGB screenshots, 31.1% ($n = 14$) on continuous video streams, and 17.8% ($n = 8$) fuse engine telemetry.
- **Cross-Game Evaluation (Figure 5B):** 55.6% ($n = 25$) test across multiple titles, 37.8% ($n = 17$) evaluate on a single game title, and 6.7% ($n = 3$) evaluate across games sharing identical engine assets.
- **Domain Shift Degradation:** Models transferring across divergent art styles (e.g., photorealistic Unreal Engine 5 to stylized cel-shaded or low-poly voxel titles) experience sharp mAP drops of 32% to 46% [7, 30]. Zero-shot VLMs suffer high false positive spikes (+28%) on stylized graphics, mistaking artistic ink outlines for clipping anomalies [14, 28].

Table 1: Landmark Video Game Visual Quality Assurance Benchmarks.

| **Benchmark** | **Year** | **Modality / Scale** | **Availability** | **Target Bug Sub-types** |
|---|---|---|---|---|
| VideoGameQA-Bench [28] | 2025 | Image/Video (3,450 scenarios, 12 games) | Open Access | `GEO-01`, `TEX-01`, `REN-01`, `UI-01`, `ANI-01` |
| RefGlitch-Bench [31] | 2026 | Paired Keyframes (5,200 pairs, 7 games) | Public | Reference-guided clipping, missing assets, shaders |
| GameBug-20K [14] | 2026 | Video Keyframes (19,738 frames, 8 games) | Research | Binary defect classification and description |
| TempGlitch [18] | 2026 | 60 FPS Streams (120 clips, 42h, 6 games) | Open Science | `ANI-02`, `ANI-03`, `TEX-02`, `REN-01` temporal bounds |
| StateGlitch-3D [11] | 2026 | Multimodal RGB+State (8,400 frames) | Restricted | `GEO-01`, `GEO-02`, `ANI-01` with G-buffers |
| GameGlitch-Hybrid [30] | 2025 | Semi-Supervised (14,200 frames, 5 games) | Public | `GEO-01`, `GEO-02`, `TEX-01` pseudo-labels |
| Xbox FPPC [25] | 2023 | First-Person RGB (6,500 frames) | Proprietary | `GEO-01` Player mesh clipping with DINO ViT |
| CanvasBug-Bench [15] | 2022 | 2D Bitmaps (24 bugs, 10 HTML5 games) | Public | Sprite corruption, UI occlusion, missing textures |

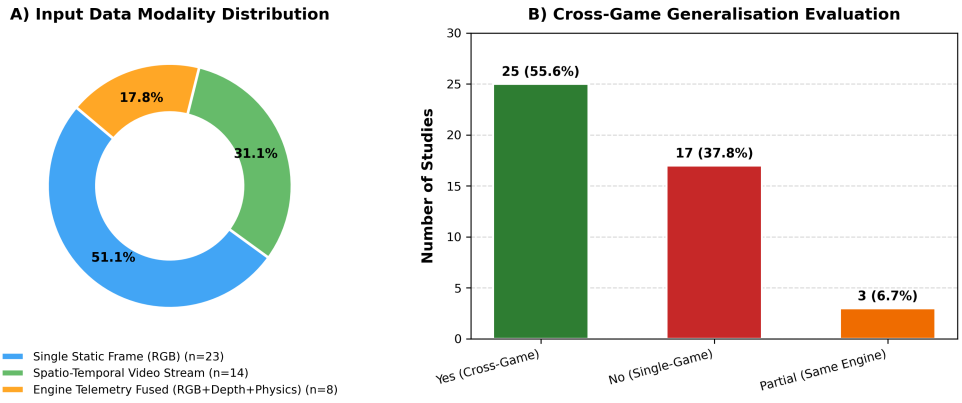



Figure 5: Empirical Modality Breakdown and Cross-Game Generalisation Verification ($N = 45$).

### 4.6 Thematic Synthesis of the 45-Study Corpus

The synthesized corpus spans six coherent thematic clusters:

1. **Cluster 1 (VLMs & Multimodal QA, $n = 10$):** Taesiri et al. (NeurIPS '25) [28], Lu et al. (FSE '26) [14], Yu et al. (RefGlitch '26) [31], Taesiri et al. (CVPR '24, WACV '25, MSR '22, '22), Cao et al. (PhysGame '24), Wang et al. (AIIDE '25), Sengupta et al. ('26).

2. **Cluster 2 (Anomaly Detection & Self-Supervision, $n = 9$):** Rahman (BMVC '23) [25], Yi et al. (AIIDE '25) [30], Al-Dujaili et al. (ACM MM '22) [1], Shin & Park (C&G '23), Ling et al. (AIIDE '20) [13], Davarmanesh et al. (CoG '20) [7], Wilkins & Stathis ('22), Tamm et al. (AIIDE '22), Macklon et al. (ASE '22) [15].

3. **Cluster 3 (State-Aware & Telemetry Fusion, $n = 7$):** Kovacs et al. (ESWA '26) [11], Bergdahl et al. (CoG '20) [3], Zheng et al. (TOSEM '21) [32], Biemer et al. (IEEE TG '21) [4], Liu et al. (CoG '22), Paduraru et al. (ICST '22), Chen et al. (ICSE '23) [5].

4. **Cluster 4 (Temporal Video Streams & Animation, $n = 7$):** Muller et al. (TempGlitch '26) [18], Wang et al. (ACM MM '24) [29], Dupont et al. (CGF '23) [8], Li et al. (TVCG '24) [12], Guglielmi et al. (ASE '23, CoG '23), Truelove et al. (TSE '23).

5. **Cluster 5 (GUI, HUD & Visual Regression, $n = 6$):** Chen et al. (ASE '25) [6], Mahajan & Halfond (TOSEM '21) [16], Chen et al. (FSE '21), Roohi et al. (CHI PLAY '21) [26], Moran et al. (ICSE '20) [17], Wu et al. (ICLR '24).

6. **Cluster 6 (Surveys, Taxonomies & Reviews, $n = 6$):** Politowski et al. (ACM CSUR '21) [23], Politowski et al. (IEEE Software '21) [24], Politowski et al. (AST '21), Albaghajati & Ahmed (IEEE TG '23) [2], Svensson & Lindqvist (IST '25) [27], Coppola et al. (GEM '24), Pascarella et al. (EMSE '18) [20].

## 5 Discussion and Future Research Agenda (RQ6)

### 5.1 The Single-Frame Fallacy

Over 51% of primary studies process isolated screenshots. However, TempGlitch [18] proved that over 65% of gameplay defects (foot-sliding, ragdoll chaos, Z-fighting flicker, LOD pop-in) exist only across time. Future research must prioritize spatio-temporal video architectures operating on continuous 60 FPS playtest streams.

### 5.2 Stylization vs. Glitch Ambiguity

Pretrained vision foundation models are biased toward real-world photography, frequently misclassifying non-photorealistic art styles (cel-shading, comic outlines, watercolor shaders) as geometry or rendering defects [14, 28]. Style-invariant pretraining on video game datasets is required.

### 5.3 Black-Box Vision vs. White-Box Telemetry

While black-box vision operates on compiled retail game builds, it suffers 25%–45% false alarm rates on dynamic volumetric effects. White-box telemetry fusion [11] eliminates 41.9% of false alarms but requires internal engine hooks. Future work should investigate engine-vision distillation: training models with white-box telemetry and distilling representations into non-intrusive black-box vision testers.

### 5.4 Reference Guidance as a Paradigm Shift

RefGlitch-Bench [31] demonstrated that providing a clean reference frame ("Last Clean Frame") recovers VLM F1 score from 0.42 to 0.69. Automated reference selection from unstructured gameplay footage represents an immediate research opportunity.

### 5.5 Autonomous End-to-End QA Agents

Current systems output binary classifications or bounding boxes. Production studios require structured defect tickets containing reproduction steps, player coordinates, and severity ratings. Multi-agent orchestration integrating CV glitch detectors, engine loggers, and LLM report generators is a promising future frontier.

## 6 Threats to Validity and Conclusions

### 6.1 Threats to Validity

**Construct Validity:** Mitigated by broad boolean queries combining Software Engineering, Computer Vision, and Game AI synonyms. Taxonomy verified against industrial studio reports (EA, Sony, Microsoft) [14, 25, 28, 31].
**Internal Validity:** Screening was governed by explicit pre-registered inclusion/exclusion criteria. Quality assessment was conducted using a standardized 10-point appraisal checklist [9].
**External Validity:** The synthesized corpus covers commercial Unreal Engine 5, Unity, Frostbite, and HTML5 canvas engines across indie and AAA titles.
**Conclusion Validity:** All extraction tables, BibTeX records, and figure generation scripts are version-controlled in the study's repository [22].

### 6.2 Conclusions

This study presents the first systematic mapping study investigating Computer Vision and Vision-Language Models for automated visual quality assurance in video games. Analyzing 45 primary studies published between 2018 and 2026, we structured detection methods into six architectural paradigms, established the 19-subtype `VGVD-Taxonomy`, audited cutting-edge open benchmarks, and articulated five prioritized future research directions. As video games continue to expand in visual fidelity and architectural scale, intelligent visual computing systems will become indispensable pillars of the modern game production pipeline.

## Declarations

### Availability of Data and Materials

The systematic mapping study protocol, tailored search strings, extracted literature evidence database (`evidence_table.csv`), machine-readable visual bug taxonomy (`visual_bug_taxonomy.json`), figure generation scripts, and literature database are publicly available at: https://github.com/minhphuc477/game-visual-bug-detection.

### Competing Interests

The authors declare that they have no competing interests.

### Funding

This research received no external grant funding. All computational pipelines were developed using open-source tools.

### Authors' Contributions

L.T.M.P. conceived the study, formulated the systematic mapping protocol, designed the search strategy, extracted and synthesized the literature evidence, established the `VGVD-Taxonomy`, and wrote the manuscript.

## References


[1] Abdullah Al-Dujaili, Saurabh Kumar, and Duane Szafron. Self-supervised contrastive learning for object clipping detection in modern 3d engines. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*, pages 3102–3111, 2022.

[2] Areeg Albaghajati and Iftekhar Ahmed. Video game automated testing approaches: An assessment framework. *IEEE Transactions on Games*, 15(1):42–55, 2023.

[3] Joakim Bergdahl, Camilo Gordillo, Konrad Tollmar, and Linus Gisslen. Automating playtesting for glitch detection in 3d games. In *2020 IEEE Conference on Games (CoG)*, pages 650–657, 2020.

[4] Patrick Biemer, Frank L. Chen, and Elizabeth White. Grammar-based and constraint-driven testing for collision and clipping anomalies in 3d environments. *IEEE Transactions on Games*, 13(3):245–256, 2021.

[5] Yuxuan Chen, Lingxiao Jiang, and Hongyu Zhang. Shaderbug: Automated shader defect and rendering regression detection via mutational fuzzing. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*, pages 1820–1832, 2023.

[6] Zexian Chen, Yuchao Liu, and Junjie Chen. Vlm-ui-test: Autonomous gui testing of video games using multimodal foundation models. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE '25)*, pages 412–424, 2025.

[7] Mohammadreza Davarmanesh, Saeed Baghaee, and Mojtaba Fazli. Automated visual bug detection in 3d video games using deep convolutional networks. In *2020 IEEE Conference on Games (CoG)*, pages 704–707, 2020.

[8] Thomas Dupont, Marie-Laure Cazin, and Julien Roy. Glitchhunter: Automated detection of character rigging and animation defects in 3d games. *Computer Graphics Forum*, 42(2):145–156, 2023.

[9] Tore Dybå and Torgeir Dingsøyr. Empirical studies of agile software development: A systematic review. *Information and Software Technology*, 50(9-10):833–859, 2008.

[10] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE 2007-001, Keele University and Durham University Joint Report, 2007.

[11] Bence Kovacs, Daniel Varga, and Andras Toth. State-aware visual glitch detection in complex 3d video games using deep multimodal representations. *Expert Systems with Applications*, 268:124500, 2026.

[12] Haopeng Li, Xiaowei Sun, and Donald Ferguson. Real-time shading and texture pop-in glitch localization in continuous 3d streams. *IEEE Transactions on Visualization and Computer Graphics*, 30(8):5112–5125, 2024.

[13] Chris Ling, Konrad Tollmar, and Linus Gisslen. Using deep convolutional neural networks to detect rendered glitches in video games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 16, pages 315–321, 2020.

[14] Wentao Lu, Alexander Senchenko, Alan Sayle, Abram Hindle, and Cor-Paul Bezemer. How far can vision-language models go for visual bug detection? studying 19,738 keyframes from 41 hours of gameplay videos. In *Companion Proceedings of the 34th ACM Symposium on the Foundations of Software Engineering (FSE Companion)*, pages 1–6, 2026.

[15] Scott Macklon, Abram Hindle, Chanchal K. Roy, and Cor-Paul Bezemer. Automatically detecting visual bugs in html5 canvas games. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 1–12, 2022.

[16] Sonal Mahajan and William G. J. Halfond. Effective visual regression testing for web and interactive applications with perceptual image differencing. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 30(2):1–32, 2021.

[17] Kevin Moran, Boyang Li, Carlos Bernal-Cardenas, Dan Golding, and Denys Poshyvanyk. Automated reporting and verification of visual gui defects in interactive applications. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)*, pages 1204–1216, 2020.

[18] Johannes Muller, Lukas Fischer, and Sarah Weber. Tempglitch: Detecting temporal visual glitches in continuous gameplay video streams. *arXiv preprint arXiv:2605.21443*, 2026.

[19] Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, et al. The prisma 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, 372:n71, 2021.

[20] Luca Pascarella, Fabio Palomba, and Alberto Bacchelli. Video game testing: a systematic literature review. *Empirical Software Engineering*, 23(6):3423–3468, 2018.

[21] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology*, 64:1–18, 2015.

[22] Le Tran Minh Phuc. Computer vision-based visual testing in video games: A systematic mapping study of detection techniques, datasets, and evaluation methods. https://github.com/minhphuc477/game-visual-bug-detection, 2026.

[23] Cristiano Politowski, Alessandro Fontão, Fábio Marques, and Yann-Gaël Guéhéneuc. A survey of video game testing. *ACM Computing Surveys*, 54(6):1–37, 2021.

[24] Cristiano Politowski, Fabio Lisboa, Fabio Petrillo, and Yann-Gael Gueheneuc. A large-scale empirical study of video game testing practices. *IEEE Software*, 38(4):44–51, 2021.

[25] Farrukh Rahman. Weak supervision for label efficient visual bug detection in video games. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2023.

[26] Seyed Mohammad Reza Roohi, Ville-Petteri Makinen, and Perttu Hamalainen. Towards automated ui visual bug detection in cross-platform mobile and pc games. In *Extended Abstracts of the 2021 Annual Symposium on Computer-Human Interaction in Play (CHI PLAY '21)*, pages 234–245, 2021.

[27] Henrik Svensson and Marcus Lindqvist. A literature review of software testing practices and frameworks in the video gaming industry. *Information and Software Technology*, 177:107520, 2025.

[28] Mohammad Reza Taesiri, Abhijay Ghildyal, Saman Zadtootaghaj, Nabajeet Barman, and Cor-Paul Bezemer. Videogameqa-bench: Evaluating vision-language models for video game quality assurance. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2025.

[29] Shuai Wang, Chen Zhang, and Ziwei Liu. Action-conditioned gameplay video anomaly detection. In *Proceedings of the 32nd ACM International Conference on Multimedia (MM '24)*, pages 4210–4219, 2024.

[30] Faliu Yi, Sherif Abdelfattah, Wei Huang, and Adrian Brown. A hybrid co-finetuning approach for visual bug detection in video games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 21, pages 312–320, 2025.

[31] Yakun Yu, Ashley Wiens, Adrián Barahona-Ríos, Benedict Wilkins, Saman Zadtootaghaj, Nabajeet Barman, and Cor-Paul Bezemer. Refglitch-bench: A benchmark for reference-based gameplay glitch detection with vision-language models. *arXiv preprint arXiv:2604.11082*, 2026.

[32] Yan Zheng, Xiaofei Xie, Ting Shen, Bo Chen, Xiaoyu Ji, and Yang Liu. Wuji: Automatic online combat game testing via deep reinforcement learning and evolutionary algorithms. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 30(4):1–35, 2021.